Foundations of Language Modeling

Chapter 4: The Mathematical Trick in Self-Attention

1 The Need for Context Aggregation

The primary limitation of the Bigram model is that tokens do not "talk" to one another; the prediction for step t relies entirely on step $t-1$. To construct a powerful language model, the representation of the token at step t must aggregate information from all previous steps $1, \dots, t$.

The simplest mechanism to aggregate information from the past is to calculate the mathematical mean (average) of the preceding vectors.

Definition 1.1 (Cumulative Average)**.** *Let $\mathbf{X} \in \mathbb{R}^{T \times C}$ be a sequence of T tokens, where each token is represented by a C-dimensional vector. We define a new sequence of aggregated vectors $\mathbf{Y} \in \mathbb{R}^{T \times C}$ such that the t-th vector $\mathbf{y}_t$ is the average of all vectors $\mathbf{x}_i$ up to and including step t:*

$$\mathbf{y}_t = \frac{1}{t} \sum_{i=1}^{t} \mathbf{x}_i \tag{1}$$

Intuition: By averaging the past, the vector $\mathbf{y}_t$ becomes a "summary" of the context up to step t. However, calculating this iteratively using `for` loops is exceptionally slow and computationally inefficient in Python and PyTorch. We need a vectorized approach.

2 The Matrix Multiplication Trick

We can perfectly execute the cumulative average operation across the entire sequence simultaneously using a single matrix multiplication with a specifically constructed lower-triangular matrix.

Theorem 2.1 (Lower-Triangular Averaging)**.** *Let $\mathbf{X} \in \mathbb{R}^{T \times C}$. Let $\mathbf{W} \in \mathbb{R}^{T \times T}$ be a lower-triangular matrix where the non-zero elements in the t-th row are uniformly $1/t$. That is:*

$$\mathbf{W}_{t,i} = \begin{cases} \frac{1}{t} & \text{if } i \leq t \\ 0 & \text{if } i > t \end{cases} \tag{2}$$

Then, the matrix multiplication $\mathbf{Y} = \mathbf{W}\mathbf{X}$ exactly yields the sequence of cumulative averages $\mathbf{Y} \in \mathbb{R}^{T \times C}$.

Remark 2.2 (Batched Matrix Multiplication)**.** *In practice, our input has a batch dimension, $\mathbf{X} \in \mathbb{R}^{B \times T \times C}$. When we multiply $\mathbf{W}$ (which can be broadcasted to $B \times T \times T$) with $\mathbf{X}$, PyTorch executes a **batched matrix multiply** (`bmm`), applying the $T \times T$ mask independently to all B sequences.*

3 The Softmax Formulation

While hardcoding the values $1/t$ works for a simple average, it is too rigid. We want a formulation where we can start with a generic mask and let the model dynamically choose the weights. We achieve this using the Softmax function.

Definition 3.1 (Causal Masking with Softmax)**.** *Let $\mathbf{M} \in \mathbb{R}^{T \times T}$ be a matrix of zeros. We apply a **causal mask** by setting the upper-triangular elements to $-\infty$:*

$$\mathbf{M}_{t,i} = \begin{cases} 0 & \text{if } i \leq t \\ -\infty & \text{if } i > t \end{cases} \tag{3}$$

Applying the Softmax function row-wise to $\mathbf{M}$ yields the averaging matrix $\mathbf{W}$:

$$\mathbf{W}_{t,*} = \mathit{softmax}(\mathbf{M}_{t,*}) \tag{4}$$

Because $\exp(-\infty) = 0$ and $\exp(0) = 1$, the Softmax evenly distributes the probability mass 1 across the unmasked elements of each row, recovering the exact $1/t$ weights from our previous theorem.

4 From Fixed Averages to Data-Dependent Attention

A simple average is a weak form of interaction; it treats all past tokens as equally important. In reality, if the current token is a verb, it might want to "attend" heavily to a noun that occurred three steps ago, and ignore conjunctions.

We evolve our Softmax formulation by replacing the zeros in the lower triangle of $\mathbf{M}$ with **data-dependent affinities**.

Definition 4.1 (Queries, Keys, and Affinities)**.** *Every token $\mathbf{x}_t \in \mathbb{R}^C$ linearly projects itself into two new vectors of dimension d_k:*

- ***Query*** $\mathbf{q}_t = \mathbf{W}_Q \mathbf{x}_t$*: "What am I looking for?"*
- ***Key*** $\mathbf{k}_t = \mathbf{W}_K \mathbf{x}_t$*: "What do I contain?"*

*The **affinity** (or attention score) between a past token i and the current token t is computed via the dot product of the current token's query and the past token's key:*

$$\mathbf{M}_{t,i} = \mathbf{q}_t \cdot \mathbf{k}_i \quad \text{for } i \leq t \tag{5}$$

Intuition: If the query of the current token aligns well with the key of a past token, their dot product will be a large positive number. When passed through the Softmax function, this token will receive a significantly higher weight than $1/t$, meaning the model is "paying attention" to it.

5 Worked Examples and Solved Exercises

5.1 Mathematical Exercise: Matrix Multiplication by Hand

Problem: Let $T = 3$ and $C = 2$. We have a sequence of 3 tokens, $\mathbf{X} \in \mathbb{R}^{3\times 2}$:

$$\mathbf{X} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \\ 1 & 3 \end{bmatrix}$$

Construct the lower-triangular averaging matrix $\mathbf{W} \in \mathbb{R}^{3\times 3}$ and calculate $\mathbf{Y} = \mathbf{W}\mathbf{X}$ by hand to prove it computes the cumulative average.

Solution: Step 1: Construct W. The row sums must equal 1, and elements where $i > t$ must be 0.

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0.333 & 0.333 & 0.333 \end{bmatrix}$$

Step 2: Compute WX.

$$\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 6 & 8 \\ 1 & 3 \end{bmatrix}$$

Row 1: $(1)(2) + 0 + 0 = 2$, and $(1)(4) + 0 + 0 = 4$. Result: $[2, 4]$. (Average of row 1).
Row 2: $(0.5)(2) + (0.5)(6) = 4$, and $(0.5)(4) + (0.5)(8) = 6$. Result: $[4, 6]$. (Average of rows 1, 2).
Row 3: $\frac{1}{3}(2 + 6 + 1) = 3$, and $\frac{1}{3}(4 + 8 + 3) = 5$. Result: $[3, 5]$. (Average of rows 1, 2, 3).

$$\mathbf{Y} = \begin{bmatrix} 2 & 4 \\ 4 & 6 \\ 3 & 5 \end{bmatrix}$$

The mathematical trick works perfectly.

5.2 Coding Example: The Softmax Causal Mask

Problem: Using PyTorch, implement the Softmax formulation of the mathematical trick for a block size of $T = 4$. Create the lower-triangular mask, apply the $-\infty$ replacement, apply Softmax, and multiply it by a random tensor $\mathbf{X}$ of shape $(B = 1, T = 4, C = 2)$.

Solution:

```
        import torch
        import torch.nn.functional as F

        B, T, C = 1, 4, 2
        x = torch.rand(B, T, C) # Random token sequence

        # 1. Create a lower triangular matrix of ones using torch.tril
        tril = torch.tril(torch.ones(T, T))
        """
        tril is now:
        [[1., 0., 0., 0.],
        [1., 1., 0., 0.],[1., 1., 1., 0.],
        [1., 1., 1., 1.]]
```

```
        """

        # 2. Create the zeros matrix and mask it with -inf
        # We use masked_fill: where tril == 0, replace with -infinity
        wei = torch.zeros((T, T))
        wei = wei.masked_fill(tril == 0, float('-inf'))
        """
        wei is now:
        [[0., -inf, -inf, -inf],[0.,   0., -inf, -inf],
        [0.,   0.,   0., -inf],[0.,   0.,   0.,   0.]]
        """

        # 3. Apply Softmax along the rows (dim=-1)
        wei = F.softmax(wei, dim=-1)
        """
        wei is now:
        [[1.00, 0.00, 0.00, 0.00],[0.50, 0.50, 0.00, 0.00],[0.33, 0.33, 0.33,
    0.00],[0.25, 0.25, 0.25, 0.25]]
        """

        # 4. Perform the batched matrix multiplication
        out = wei @ x # PyTorch treats this as (B, T, T) @ (B, T, C) -> (B, T, C)

        print("Original X:\n", x)
        print("Averaged Output:\n", out)

```